Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №5

з дисципліни

«Об’єктно-орієнтоване програмування»

на тему

«Розробка багатовіконного інтерфейсу користувача для графічного редактора об’єктів»

Виконав:
студент групи ІМ-51
Демченко Олександр Стефанович
Номер у списку - 10

Перевірив:
Рекечинський Д.О.

Київ 2026

Мета: Мета роботи – отримати вміння та навички програмувати багатовіконний інтерфейс програми в об’єктно-орієнтованому стилі.

Завдання

1. Створити у середовищі розробки проект з ім’ям Lab5.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. За потреби налагодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.
6. Оформити звіт.

Варіант завдання

Ж = 10

Перелік фігур у меню «Об'єкти»: Крапка, Лінія, Прямокутник, Еліпс, Лінія з кружечками, Куб.

MyEditor: на основі класичної реалізації Singleton.

Вихідний текст головного файлу

MainFrame.java

```
package Lab5;

import Lab5.shapes.*;
import Lab5.editor.*;
import Lab5.ui.*;

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.io.File;

public class MainFrame extends JFrame {
   public static final int MODE_POINT = 1, MODE_LINE = 2, MODE_RECTANGLE = 3, MODE_ELLIPSE = 4,
MODE_LINE_OO = 5, MODE_CUBE = 6;
   private int currentMode = MODE_POINT;

   private MyEditor editor;
   private MyTable tableWindow;

   private JRadioButtonMenuItem itemPoint, itemLine, itemRectangle, itemEllipse, itemLineOO, itemCube;
```

```
private JToggleButton btnPoint, btnLine, btnRectangle, btnEllipse, btnLineOO, btnCube;
private CanvasPanel canvasPanel;

public MainFrame() {
  setSize(850, 650);
  setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
  setLocationRelativeTo(null);
  setLayout(new BorderLayout());

  //Отримання екземпляра Singleton
  editor = MyEditor.getInstance();

  //Створення вікна таблиці
  tableWindow = new MyTable(this);

  //Callback при зміні даних MyEditor
  editor.setOnDataChangedCallback(() -> {
    tableWindow.updateTable(editor.getShapes());
    if (canvasPanel != null) canvasPanel.repaint();
  });

  //Реалізація Listener для бонусів 1 та 2
  tableWindow.setTableListener(new TableListener() {
    @Override
    public void onRowSelected(int rowIndex) {
      editor.selectShape(rowIndex);
      canvasPanel.repaint();
    }

    @Override
    public void onRowDeleted(int rowIndex) {
      editor.removeShape(rowIndex);
    }
  });

  initMenu();
  initToolBar();

  canvasPanel = new CanvasPanel();
  add(canvasPanel, BorderLayout.CENTER);

  setMode(MODE_POINT);

  //Автоматичне закриття таблиці під час закриття програми
  addWindowListener(new WindowAdapter() {
    @Override
    public void windowClosing(WindowEvent e) {
      if (tableWindow != null) tableWindow.dispose();
    }
  });
```

```
    }

    private void setMode(int mode) {
        this.currentMode = mode;
        setTitle("Лабораторна робота №5 - [Обрано об'єкт: " + switch (mode) {
            case MODE_POINT -> "Крапка"; case MODE_LINE -> "Лінія"; case MODE_RECTANGLE ->
"Прямокутник";
            case MODE_ELLIPSE -> "Еліпс"; case MODE_LINE_OO -> "Лінія з кружечками"; case MODE_CUBE -
> "Каркас куба"; default -> "";
        } + "]");

        switch (mode) {
            case MODE_POINT -> { btnPoint.setSelected(true); itemPoint.setSelected(true); }
            case MODE_LINE -> { btnLine.setSelected(true); itemLine.setSelected(true); }
            case MODE_RECTANGLE -> { btnRectangle.setSelected(true); itemRectangle.setSelected(true); }
            case MODE_ELLIPSE -> { btnEllipse.setSelected(true); itemEllipse.setSelected(true); }
            case MODE_LINE_OO -> { btnLineOO.setSelected(true); itemLineOO.setSelected(true); }
            case MODE_CUBE -> { btnCube.setSelected(true); itemCube.setSelected(true); }
        }
    }

    private void initMenu() {
        JMenuBar menuBar = new JMenuBar();

        JMenu menuFile = new JMenu("Файл");
        JMenuItem itemSave = new JMenuItem("Зберегти у CSV...");
        itemSave.addActionListener(e -> saveFile());
        JMenuItem itemLoad = new JMenuItem("Завантажити з CSV...");
        itemLoad.addActionListener(e -> loadFile());
        JMenuItem itemExit = new JMenuItem("Вихід");
        itemExit.addActionListener(e -> System.exit(0));
        menuFile.add(itemSave);
        menuFile.add(itemLoad);
        menuFile.addSeparator();
        menuFile.add(itemExit);

        JMenu menuObjects = new JMenu("Об'єкти");
        ButtonGroup menuGroup = new ButtonGroup();
        itemPoint = new JRadioButtonMenuItem("Крапка");
        itemLine = new JRadioButtonMenuItem("Лінія");
        itemRectangle = new JRadioButtonMenuItem("Прямокутник");
        itemEllipse = new JRadioButtonMenuItem("Еліпс");
        itemLineOO = new JRadioButtonMenuItem("Лінія з кружечками");
        itemCube = new JRadioButtonMenuItem("Каркас куба");

        itemPoint.addActionListener(e -> setMode(MODE_POINT));
        itemLine.addActionListener(e -> setMode(MODE_LINE));
        itemRectangle.addActionListener(e -> setMode(MODE_RECTANGLE));
        itemEllipse.addActionListener(e -> setMode(MODE_ELLIPSE));
        itemLineOO.addActionListener(e -> setMode(MODE_LINE_OO));
```

```
        itemCube.addActionListener(e -> setMode(MODE_CUBE));

        JRadioButtonMenuItem[] items = {itemPoint, itemLine, itemRectangle, itemEllipse, itemLineOO,
itemCube};
        for (JRadioButtonMenuItem item : items) { menuGroup.add(item); menuObjects.add(item); }

        JMenu menuView = new JMenu("Вигляд");
        JMenuItem itemTable = new JMenuItem("Таблиця об'єктів");
        itemTable.addActionListener(e -> tableWindow.setVisible(!tableWindow.isVisible()));
        menuView.add(itemTable);

        JMenu menuHelp = new JMenu("Довідка");
        JMenuItem itemAbout = new JMenuItem("Про програму");
        itemAbout.addActionListener(e -> JOptionPane.showMessageDialog(this, "Графічний редактор.
Лабораторна робота №5"));
        menuHelp.add(itemAbout);

        menuBar.add(menuFile);
        menuBar.add(menuObjects);
        menuBar.add(menuView);
        menuBar.add(menuHelp);
        setJMenuBar(menuBar);
    }

    private void saveFile() {
        JFileChooser fc = new JFileChooser();
        if (fc.showSaveDialog(this) == JFileChooser.APPROVE_OPTION) {
            try {
                editor.saveToCSV(fc.getSelectedFile());
                JOptionPane.showMessageDialog(this, "Файл успішно збережено!");
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Помилка збереження: " + ex.getMessage(), "Помилка",
JOptionPane.ERROR_MESSAGE);
            }
        }
    }

    private void loadFile() {
        JFileChooser fc = new JFileChooser();
        if (fc.showOpenDialog(this) == JFileChooser.APPROVE_OPTION) {
            try {
                editor.loadFromCSV(fc.getSelectedFile());
                JOptionPane.showMessageDialog(this, "Файл успішно завантажено!");
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Помилка завантаження: " + ex.getMessage(),
"Помилка", JOptionPane.ERROR_MESSAGE);
            }
        }
    }
```

```
  private ImageIcon loadScaledIcon(String path, int width, int height) {
    java.net.URL imgURL = getClass().getResource(path);
    return (imgURL != null) ? new ImageIcon(new
ImageIcon(imgURL).getImage().getScaledInstance(width, height, Image.SCALE_SMOOTH)) : null;
  }

  private void initToolBar() {
    JToolBar toolBar = new JToolBar();
    toolBar.setFloatable(false);
    ButtonGroup toolGroup = new ButtonGroup();

    btnPoint = new JToggleButton(loadScaledIcon("/Lab3/icons/point.png", 20, 20));
    btnLine = new JToggleButton(loadScaledIcon("/Lab3/icons/line.png", 20, 20));
    btnRectangle = new JToggleButton(loadScaledIcon("/Lab3/icons/rect.png", 20, 20));
    btnEllipse = new JToggleButton(loadScaledIcon("/Lab3/icons/ellipse.png", 20, 20));
    btnLineOO = new JToggleButton(loadScaledIcon("/Lab3/icons/LineCircles.jpg", 20, 20));
    btnCube = new JToggleButton(loadScaledIcon("/Lab3/icons/cube.png", 20, 20));

    btnPoint.setToolTipText("Крапка"); btnLine.setToolTipText("Лінія");
btnRectangle.setToolTipText("Прямокутник");
    btnEllipse.setToolTipText("Еліпс"); btnLineOO.setToolTipText("Лінія з кружечками");
btnCube.setToolTipText("Каркас куба");

    btnPoint.addActionListener(e -> setMode(MODE_POINT)); btnLine.addActionListener(e ->
setMode(MODE_LINE));
    btnRectangle.addActionListener(e -> setMode(MODE_RECTANGLE)); btnEllipse.addActionListener(e
-> setMode(MODE_ELLIPSE));
    btnLineOO.addActionListener(e -> setMode(MODE_LINE_OO)); btnCube.addActionListener(e ->
setMode(MODE_CUBE));

    JToggleButton[] buttons = {btnPoint, btnLine, btnRectangle, btnEllipse, btnLineOO, btnCube};
    for (JToggleButton btn : buttons) { toolGroup.add(btn); toolBar.add(btn); }
    add(toolBar, BorderLayout.NORTH);
  }

  private class CanvasPanel extends JPanel {
    private int currentX, currentY;
    public CanvasPanel() {
      setBackground(Color.WHITE);
      MouseAdapter mouseHandler = new MouseAdapter() {
        @Override
        public void mousePressed(MouseEvent e) {
          currentX = e.getX(); currentY = e.getY();
          editor.startShape(currentMode, currentX, currentY);
        }
        @Override
        public void mouseDragged(MouseEvent e) {
          currentX = e.getX(); currentY = e.getY();
          editor.dragShape(currentX, currentY); repaint();
        }
```

```
            @Override
            public void mouseReleased(MouseEvent e) {
               currentX = e.getX(); currentY = e.getY();
               if (!editor.releaseShape(currentX, currentY)) {
                  JOptionPane.showMessageDialog(MainFrame.this, "Переповнення масиву об'єктів (111)!",
"Помилка", JOptionPane.ERROR_MESSAGE);
               }
               repaint();
            }
         };
         addMouseListener(mouseHandler); addMouseMotionListener(mouseHandler);
      }
      @Override
      protected void paintComponent(Graphics g) {
         super.paintComponent(g);
         editor.drawAll(g, currentX, currentY);
      }
   }

   public static void main(String[] args) {
      SwingUtilities.invokeLater(() -> new MainFrame().setVisible(true));
   }
}
```

MyEditor.java

```
package Lab5.editor;

import Lab5.MainFrame;
import Lab5.shapes.*;
import java.awt.Graphics;
import java.io.*;
import java.util.ArrayList;
import java.util.List;

public class MyEditor {
   private static MyEditor instance;
   private static final int MAX_SHAPES = 111;
   private final List<Shape> shapes = new ArrayList<>();

   private Shape currentShape = null;
   private int startX, startY;
   private boolean isDragging = false;
   private Runnable onDataChangedCallback;

   //Приватний конструктор Singleton
   private MyEditor() {}
```

```
public static synchronized MyEditor getInstance() {
   if (instance == null) {
      instance = new MyEditor();
   }
   return instance;
}

public void setOnDataChangedCallback(Runnable callback) {
   this.onDataChangedCallback = callback;
}

private void notifyDataChanged() {
   if (onDataChangedCallback != null) {
      onDataChangedCallback.run();
   }
}

public List<Shape> getShapes() {
   return shapes;
}

public boolean addShape(Shape shape) {
   if (shapes.size() < MAX_SHAPES) {
      shapes.add(shape);
      notifyDataChanged();
      return true;
   }
   return false;
}

public void selectShape(int index) {
   for (int i = 0; i < shapes.size(); i++) {
      shapes.get(i).setSelected(i == index);
   }
}

public void removeShape(int index) {
   if (index >= 0 && index < shapes.size()) {
      shapes.remove(index);
      notifyDataChanged();
   }
}

public void startShape(int mode, int x, int y) {
   startX = x; startY = y;
   currentShape = switch (mode) {
      case MainFrame.MODE_POINT -> new PointShape();
      case MainFrame.MODE_LINE -> new LineShape();
      case MainFrame.MODE_RECTANGLE -> new RectangleShape();
      case MainFrame.MODE_ELLIPSE -> new EllipseShape();
```

```
            case MainFrame.MODE_LINE_OO -> new LineCirclesShape();
            case MainFrame.MODE_CUBE -> new CubeShape();
            default -> null;
        };
        if (currentShape != null) {
            currentShape.setPoints(startX, startY, startX, startY);
            isDragging = true;
        }
    }

    public void dragShape(int x, int y) {
        if (isDragging && currentShape != null) {
            currentShape.setPoints(startX, startY, x, y);
        }
    }

    public boolean releaseShape(int x, int y) {
        if (isDragging && currentShape != null) {
            currentShape.setPoints(startX, startY, x, y);
            boolean added = addShape(currentShape);
            currentShape = null;
            isDragging = false;
            return added;
        }
        return true;
    }

    public void drawAll(Graphics g, int currentX, int currentY) {
        for (Shape shape : shapes) {
            shape.show(g);
        }
        if (isDragging && currentShape != null) {
            currentShape.trail(g, currentX, currentY);
        }
    }

    //Збереження в CSV
    public void saveToCSV(File file) throws IOException {
        try (PrintWriter writer = new PrintWriter(new FileWriter(file))) {
            for (Shape shape : shapes) {
                writer.println(shape.getName() + "," + shape.getX1() + "," + shape.getY1() + "," + shape.getX2() +
"," + shape.getY2());
            }
        }
    }

    //Завантаження із CSV
    public void loadFromCSV(File file) throws IOException {
        shapes.clear();
        try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
```

```
            String line;
            while ((line = reader.readLine()) != null) {
                String[] parts = line.split(",");
                if (parts.length == 5) {
                    String name = parts[0];
                    int x1 = Integer.parseInt(parts[1]);
                    int y1 = Integer.parseInt(parts[2]);
                    int x2 = Integer.parseInt(parts[3]);
                    int y2 = Integer.parseInt(parts[4]);

                    Shape shape = switch (name) {
                        case "Крапка" -> new PointShape();
                        case "Лінія" -> new LineShape();
                        case "Прямокутник" -> new RectangleShape();
                        case "Еліпс" -> new EllipseShape();
                        case "Лінія з кружечками" -> new LineCirclesShape();
                        case "Каркас куба" -> new CubeShape();
                        default -> null;
                    };

                    if (shape != null) {
                        shape.setPoints(x1, y1, x2, y2);
                        shapes.add(shape);
                    }
                }
            }
        }
        notifyDataChanged();
    }
}
```

MyTable.java

```
package Lab5.ui;

import Lab5.shapes.Shape;
import Lab5.editor.TableListener;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;

public class MyTable extends JDialog {
    private JTable table;
    private DefaultTableModel tableModel;
    private TableListener listener;

    public MyTable(Frame owner) {
        super(owner, "Таблиця графічних об'єктів", false); //false = Немодальне вікно
        setSize(450, 300);
```

```
    setLocationRelativeTo(owner);
    setLayout(new BorderLayout());

    String[] columns = {"Назва", "x1", "y1", "x2", "y2"};
    tableModel = new DefaultTableModel(columns, 0) {
        @Override
        public boolean isCellEditable(int row, int column) {
            return false;
        }
    };

    table = new JTable(tableModel);
    add(new JScrollPane(table), BorderLayout.CENTER);

    //Обробка виділення рядка
    table.getSelectionModel().addListSelectionListener(e -> {
        if (!e.getValueIsAdjusting() && listener != null) {
            int selectedRow = table.getSelectedRow();
            listener.onRowSelected(selectedRow);
        }
    });

    //Панель з кнопкою видалення
    JPanel bottomPanel = new JPanel();
    JButton btnDelete = new JButton("Видалити об'єкт");
    btnDelete.addActionListener(e -> {
        int selectedRow = table.getSelectedRow();
        if (selectedRow != -1 && listener != null) {
            listener.onRowDeleted(selectedRow);
        }
    });
    bottomPanel.add(btnDelete);
    add(bottomPanel, BorderLayout.SOUTH);
}

public void setTableListener(TableListener listener) {
    this.listener = listener;
}

public void updateTable(java.util.List<Shape> shapes) {
    tableModel.setRowCount(0);
    for (Shape shape : shapes) {
        if (shape != null) {
            tableModel.addRow(new Object[]{
                    shape.getName(),
                    shape.getX1(),
                    shape.getY1(),
                    shape.getX2(),
                    shape.getY2()
            });
```

```
        }
      }
    }
}
```

TableListener.java

```
package Lab5.editor;

public interface TableListener {
    void onRowSelected(int rowIndex);
    void onRowDeleted(int rowIndex);
}
```

Shape.java

```
package Lab5.shapes;

import java.awt.BasicStroke;
import java.awt.Color;
import java.awt.Graphics;
import java.awt.Graphics2D;

public abstract class Shape {
    protected int x1, y1, x2, y2;
    protected boolean selected = false; //Позначка виділення об'єкта в таблиці

    public void setPoints(int x1, int y1, int x2, int y2) {
        this.x1 = x1;
        this.y1 = y1;
        this.x2 = x2;
        this.y2 = y2;
    }

    public int getX1() { return x1; }
    public int getY1() { return y1; }
    public int getX2() { return x2; }
    public int getY2() { return y2; }

    public void setSelected(boolean selected) {
        this.selected = selected;
    }

    public boolean isSelected() {
        return selected;
```

```
    }

    protected void applyOutlineStyle(Graphics2D g2d, Color originalColor) {
        if (selected) {
            g2d.setColor(Color.RED);
            g2d.setStroke(new BasicStroke(3));
        } else {
            g2d.setColor(originalColor);
            g2d.setStroke(new BasicStroke(1));
        }
    }

    public abstract String getName();
    public abstract void show(Graphics g);
    public abstract void trail(Graphics g, int currentX, int currentY);
}
```

PointShape.java

```
package Lab5.shapes;

import java.awt.Color;
import java.awt.Graphics;
import java.awt.Graphics2D;

public class PointShape extends Shape {

    @Override
    public String getName() {
        return "Крапка";
    }

    @Override
    public void show(Graphics g) {
        Graphics2D g2d = (Graphics2D) g;
        int r = selected ? 6 : 3;
        applyOutlineStyle(g2d, Color.BLACK);
        g2d.fillOval(x1 - r, y1 - r, 2 * r, 2 * r);
    }

    @Override
    public void trail(Graphics g, int currentX, int currentY) {
        g.setColor(Color.BLACK);
        g.fillRect(currentX, currentY, 2, 2);
    }
}
```

LineShape.java

```
package Lab5.shapes;
import java.awt.BasicStroke;
import java.awt.Color;
import java.awt.Graphics;
import java.awt.Graphics2D;

public class LineShape extends Shape {

   @Override
   public String getName() {
      return "Лінія";
   }

   @Override
   public void show(Graphics g) {
      Graphics2D g2d = (Graphics2D) g;
      applyOutlineStyle(g2d, Color.BLACK);
      g2d.drawLine(x1, y1, x2, y2);
   }

   @Override
   public void trail(Graphics g, int currentX, int currentY) {
      //"Гумовий" слід: пунктирна лінія чорного кольору
      Graphics2D g2d = (Graphics2D) g.create();
      g2d.setColor(Color.BLACK);
      float[] dash = {5.0f, 5.0f};
      g2d.setStroke(new BasicStroke(1.0f, BasicStroke.CAP_BUTT, BasicStroke.JOIN_MITER, 10.0f, dash,
0.0f));
      g2d.drawLine(x1, y1, currentX, currentY);
      g2d.dispose();
   }
}
```

RectangleShape.java

```
package Lab5.shapes;

import java.awt.BasicStroke;
import java.awt.Color;
import java.awt.Graphics;
import java.awt.Graphics2D;

public class RectangleShape extends Shape {
   @Override
   public String getName() {
```

```
    return "Прямокутник";
  }

  @Override
  public void show(Graphics g) {
    Graphics2D g2d = (Graphics2D) g;

    int minX = Math.min(x1, x2);
    int minY = Math.min(y1, y2);
    int width = Math.abs(x2 - x1);
    int height = Math.abs(y2 - y1);

    if (width > 0 && height > 0) {
      g2d.setColor(Color.GRAY);
      g2d.fillRect(minX, minY, width, height);

      applyOutlineStyle(g2d, Color.BLACK);
      g2d.drawRect(minX, minY, width, height);
    }
  }

  @Override
  public void trail(Graphics g, int currentX, int currentY) {
    //Пунктирний чорний "гумовий" слід від центру
    Graphics2D g2d = (Graphics2D) g.create();
    g2d.setColor(Color.BLACK);
    float[] dash = {5.0f, 5.0f};
    g2d.setStroke(new BasicStroke(1.0f, BasicStroke.CAP_BUTT, BasicStroke.JOIN_MITER, 10.0f, dash,
0.0f));

    int minX = Math.min(x1, currentX);
    int minY = Math.min(y1, currentY);
    int width = Math.abs(currentX - x1);
    int height = Math.abs(currentY - y1);

    g2d.drawRect(minX, minY, width, height);
    g2d.dispose();
  }
}
```

EllipseShape.java

```
package Lab5.shapes;

import java.awt.BasicStroke;
import java.awt.Color;
import java.awt.Graphics;
import java.awt.Graphics2D;
```

```
public class EllipseShape extends Shape {

    @Override
    public String getName() {
        return "Еліпс";
    }

    @Override
    public void show(Graphics g) {
        Graphics2D g2d = (Graphics2D) g;

        int minX = Math.min(x1, x2);
        int minY = Math.min(y1, y2);
        int width = Math.abs(x2 - x1);
        int height = Math.abs(y2 - y1);

        g2d.setColor(Color.WHITE);
        g2d.fillOval(minX, minY, width, height);

        applyOutlineStyle(g2d, Color.BLACK);
        g2d.drawOval(minX, minY, width, height);
    }

    @Override
    public void trail(Graphics g, int currentX, int currentY) {
        //Пунктирний чорний "гумовий" слід по двох кутах
        Graphics2D g2d = (Graphics2D) g.create();
        g2d.setColor(Color.BLACK);
        float[] dash = {5.0f, 5.0f};
        g2d.setStroke(new BasicStroke(1.0f, BasicStroke.CAP_BUTT, BasicStroke.JOIN_MITER, 10.0f, dash,
0.0f));

        int x = Math.min(x1, currentX);
        int y = Math.min(y1, currentY);
        int width = Math.abs(currentX - x1);
        int height = Math.abs(currentY - y1);

        g2d.drawOval(x, y, width, height);
        g2d.dispose();
    }
}
```

LineCirclesShape.java

```
package Lab5.shapes;

import java.awt.Graphics;
```

```
public class LineCirclesShape extends Shape {
   private final LineShape line = new LineShape();
   private final EllipseShape circle1 = new EllipseShape();
   private final EllipseShape circle2 = new EllipseShape();
   private static final int RADIUS = 6;

   @Override
   public void setPoints(int x1, int y1, int x2, int y2) {
      super.setPoints(x1, y1, x2, y2);
      line.setPoints(x1, y1, x2, y2);
      circle1.setPoints(x1 - RADIUS, y1 - RADIUS, x1 + RADIUS, y1 + RADIUS);
      circle2.setPoints(x2 - RADIUS, y2 - RADIUS, x2 + RADIUS, y2 + RADIUS);
   }

   @Override
   public String getName() {
      return "Лінія з кружечками";
   }

   @Override
   public void setSelected(boolean selected) {
      super.setSelected(selected);
      line.setSelected(selected);
      circle1.setSelected(selected);
      circle2.setSelected(selected);
   }

   @Override
   public void show(Graphics g) {
      line.show(g);
      circle1.show(g);
      circle2.show(g);
   }

   @Override
   public void trail(Graphics g, int currentX, int currentY) {
      line.trail(g, currentX, currentY);
   }
}
```

CubeShape.java

```
package Lab5.shapes;

import java.awt.Graphics;

public class CubeShape extends Shape {
   private final RectangleShape rectFront = new RectangleShape();
```

```
private final RectangleShape rectBack = new RectangleShape();
private final LineShape line1 = new LineShape();
private final LineShape line2 = new LineShape();
private final LineShape line3 = new LineShape();
private final LineShape line4 = new LineShape();

@Override
public void setPoints(int x1, int y1, int x2, int y2) {
   super.setPoints(x1, y1, x2, y2);

   //Нормалізація координат прямокутника
   int minX = Math.min(x1, x2);
   int maxX = Math.max(x1, x2);
   int minY = Math.min(y1, y2);
   int maxY = Math.max(y1, y2);

   //Зсув для об'єму
   int dx = (maxX - minX) / 3;
   int dy = (maxY - minY) / 3;

   //Передній прямокутник
   int fx1 = minX,      fy1 = minY + dy;
   int fx2 = maxX - dx, fy2 = maxY;
   rectFront.setPoints(fx1, fy1, fx2, fy2);

   //Задній прямокутник
   int bx1 = minX + dx,  by1 = minY;
   int bx2 = maxX,      by2 = maxY - dy;
   rectBack.setPoints(bx1, by1, bx2, by2);

   //Чотири сполучні лінії між вершинами двох прямокутників
   line1.setPoints(fx1, fy1, bx1, by1);
   line2.setPoints(fx2, fy1, bx2, by1);
   line3.setPoints(fx2, fy2, bx2, by2);
   line4.setPoints(fx1, fy2, bx1, by2);
}

@Override
public String getName() {
   return "Каркас куба";
}

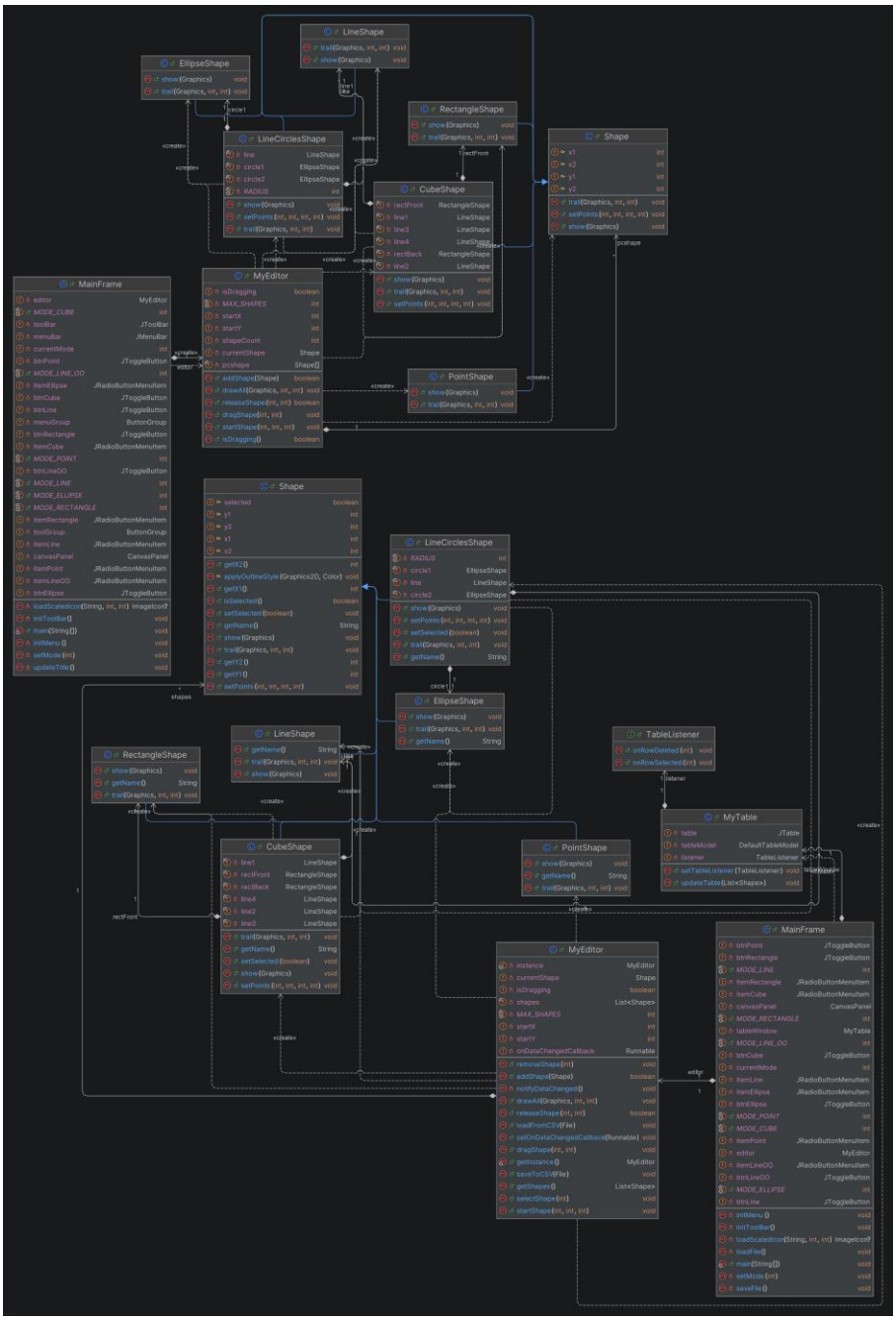
@Override
public void setSelected(boolean selected) {
   super.setSelected(selected);
   rectFront.setSelected(selected);
   rectBack.setSelected(selected);
   line1.setSelected(selected);
   line2.setSelected(selected);
   line3.setSelected(selected);
```

```
        line4.setSelected(selected);
    }

    @Override
    public void show(Graphics g) {
        rectBack.show(g);
        rectFront.show(g);
        line1.show(g);
        line2.show(g);
        line3.show(g);
        line4.show(g);
    }

    @Override
    public void trail(Graphics g, int currentX, int currentY) {
        setPoints(x1, y1, currentX, currentY);
        show(g);
    }
}
```

Діаграма успадкування класів



Для кращого перегляду діаграму розміщено на github -> [Diagram]

Ілюстрації роботи програми

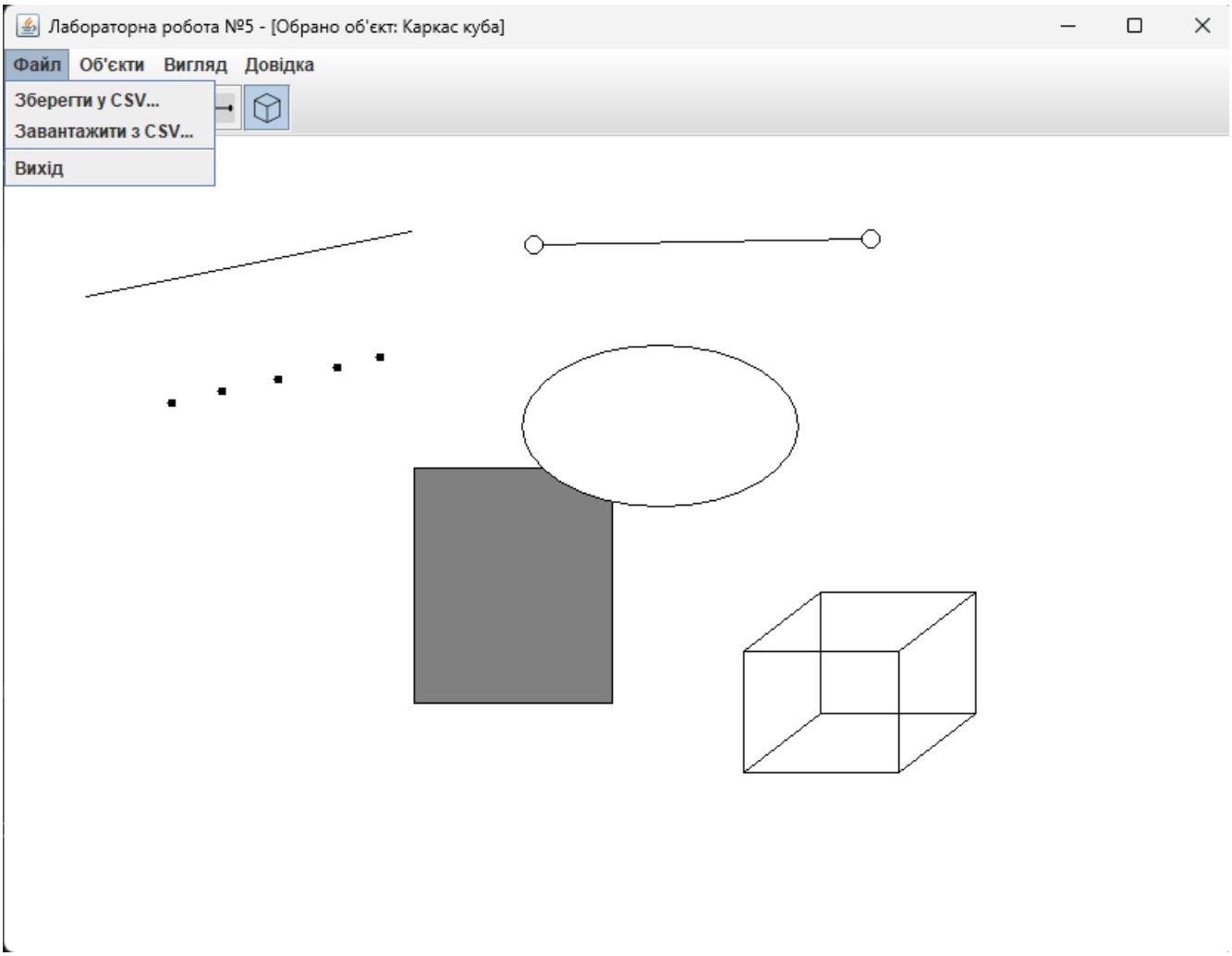


Рис.1 - нові функції збереження/завантаження CSV

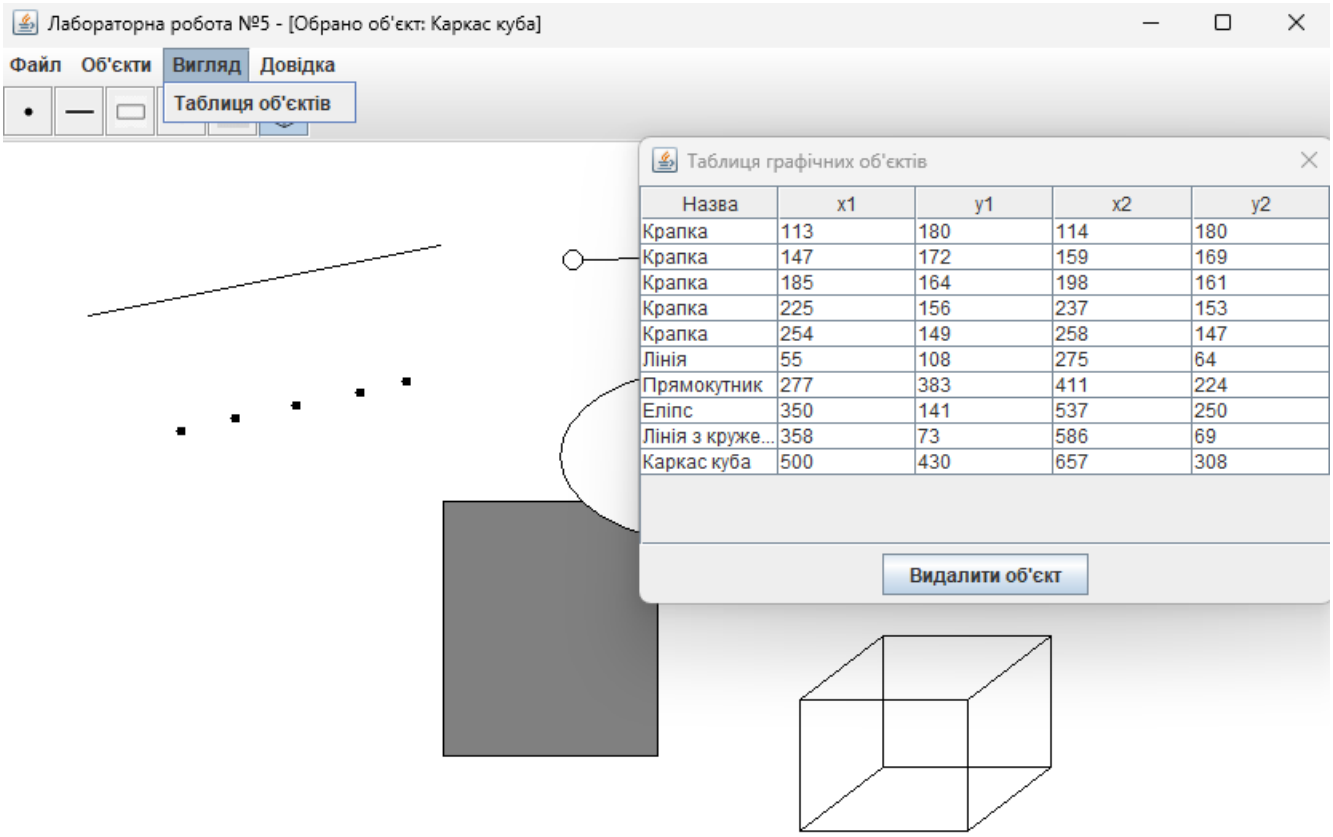


Рис.2 - таблиця об’єктів

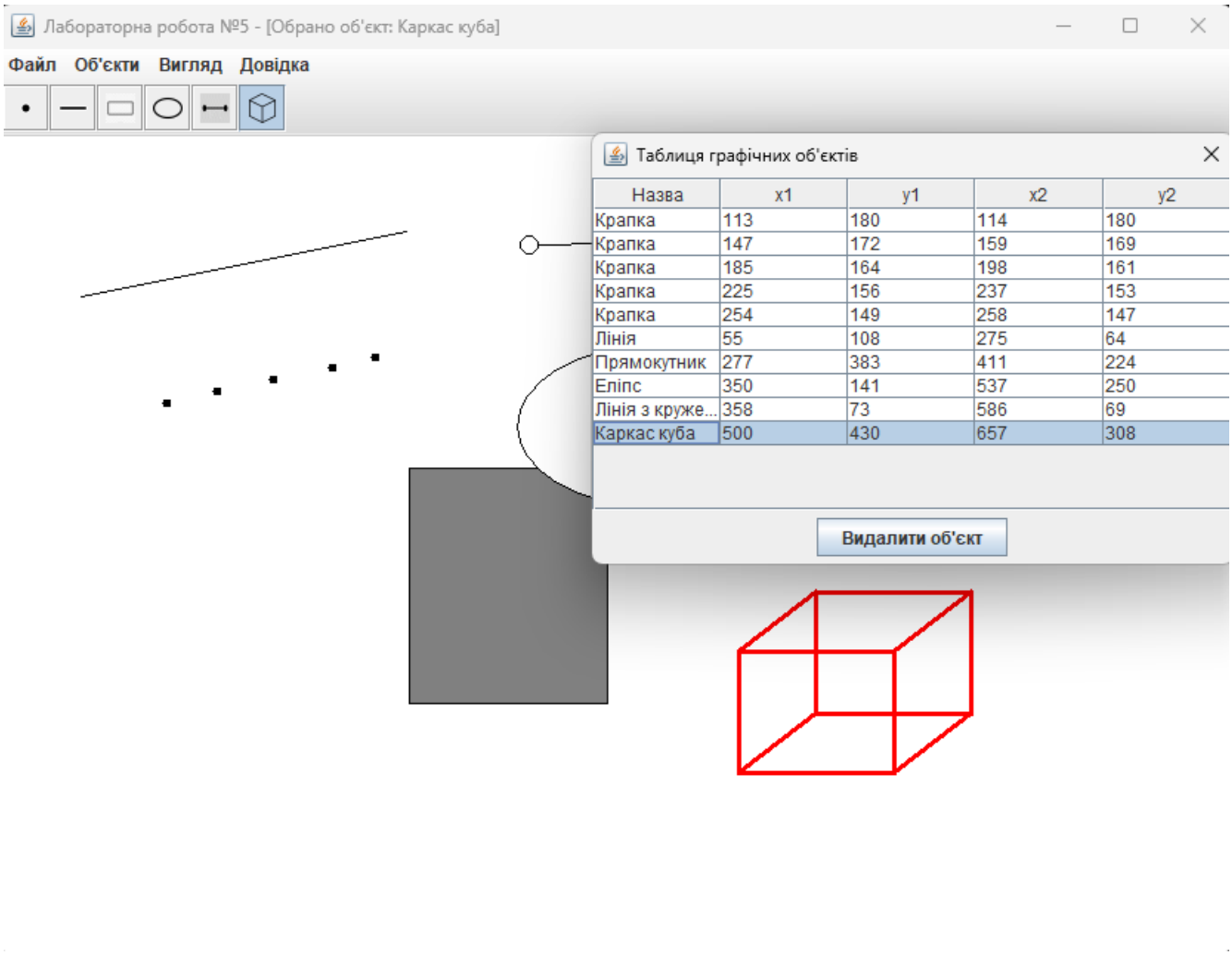


Рис.3 - приклад виділення об’єкта

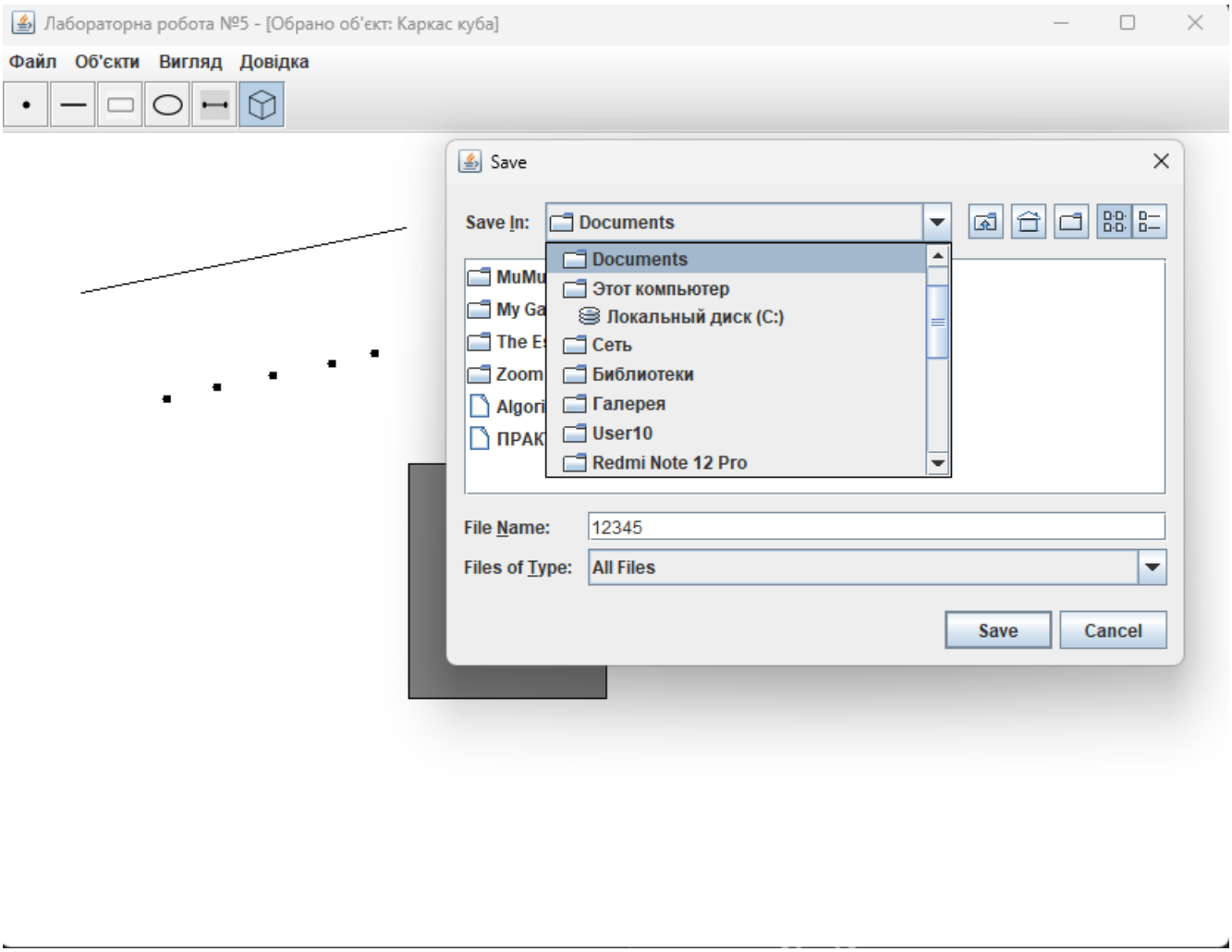


Рис.4 - функція збереження файла у CSV форматі

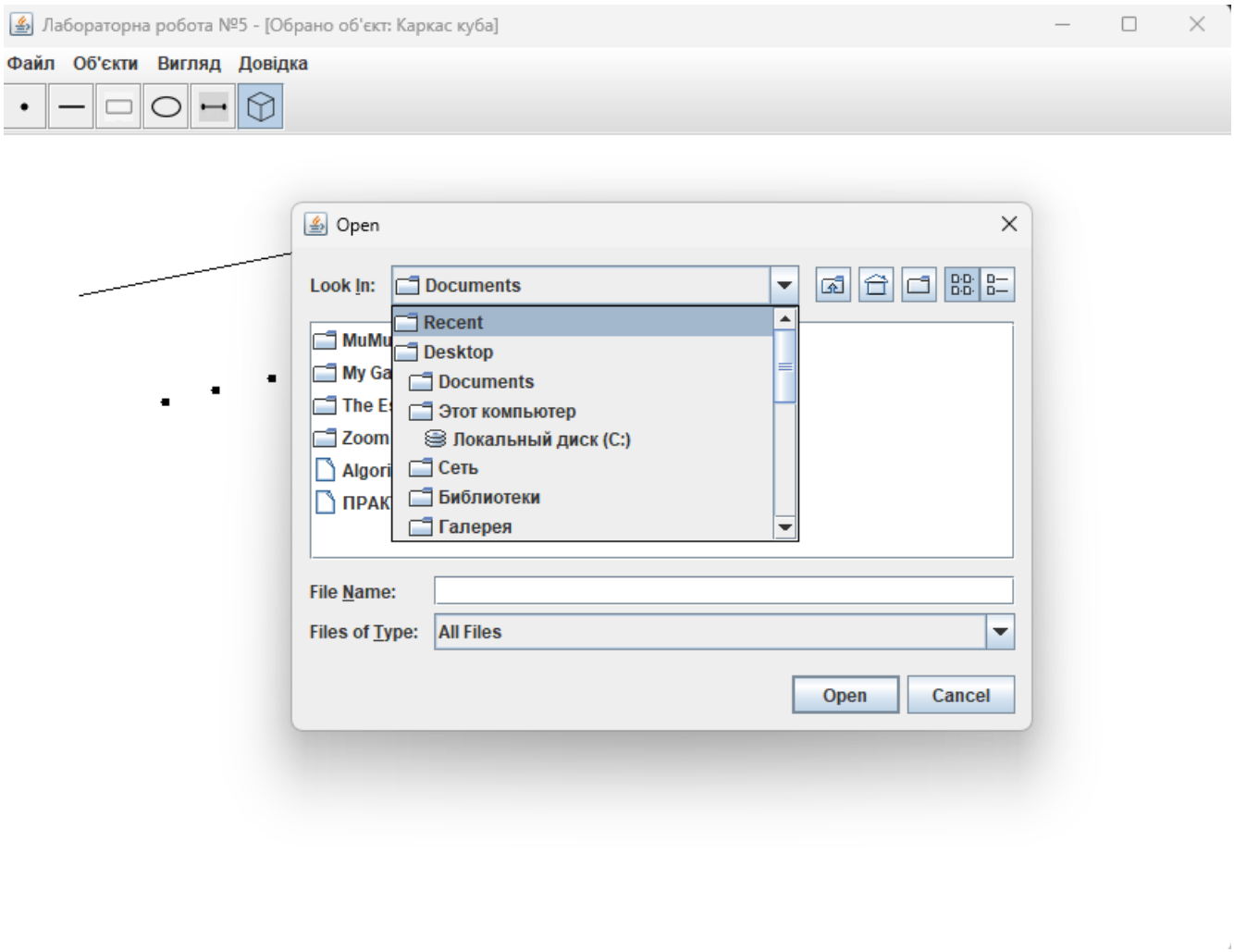


Рис.5 - функція завантаження файлу з розширенням CSV

Висновок

Під час виконання лабораторної роботи №5 було вдосконалено архітектуру графічного редактора шляхом переведення класу `MyEditor` на класичний паттерн Singleton ($Ж = 10$) та додавання немодального діалогового вікна `MyTable` для відображення списку об'єктів у вигляді таблиці. Для дотримання принципу послаблення зв'язності (Loose Coupling) взаємодію між вікном таблиці та редактором було реалізовано через інтерфейс-слухач `TableListener` (паттерн Observer). У результаті було впроваджено функціонал збереження й завантаження даних у текстовий CSV-файл, а також повністю реалізовано всі бонусні завдання: динамічне видалення об'єктів з таблиці та їхнє візуальне підсвічування червоним контуром на полотні зі збереженням оригінальних кольорів і заповнення фігур.